目录

图论

最小生成树 - Prim 算法 (堆优化版)

算法说明

用途：求无向图的最小生成树 (MST) 的权值之和。

时间复杂度：O(ElogE) 或 O(ElogV)。

适用场景：适合稠密图。对于稀疏图，Kruskal 算法通常代码更短，但堆优化的 Prim 性能也十分优秀。

图论 - 最小生成树 (MST) - Kruskal 算法

算法说明 * **用途**：在无向连通图中寻找一棵包含所有顶点且边权之和最小的树。

* **时间复杂度**：$O(M \log M)$（M 为边数，瓶颈在边排序）。

* **适用场景**：稀疏图（边数较少）。基于贪心思想，配合并查集，代码极简，是比赛中最常用的求 MST 算法。

* **注意事项**：

1. 必须在读入边之前或之后，初始化并查集 `p[i] = i`。
2. 如果最终加入的边数不等于 $N-1$，说明图不连通，不存在最小生成树。
3. 累加的边权总和极易爆 `int`，务必使用 `long long` 存储 `res`。

最小生成数 Kruskal.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define endl '\n'
// 将边权 w 放在第一个位置，利用 tuple 默认字典序实现自带按边权排序
using Edge = tuple<int, int, int>;
void solve()
{
    int n, m;
    cin >> n >> m;
    vector<Edge> edges(m);
    vector<int> p(n + 1); // 并查集父节点数组（原 fe）
    // lambda 并查集寻根函数（路径压缩）
    function<int(int)> find = [&](int x)->int
        {
            if(p[x] != x)p[x] = find(p[x]);
            return p[x];
        };

    // 1. 初始化并查集
    for(int i = 1;i <= n;i++)p[i] = i;
    // 2. 读入边，存入顺序为 {边权，端点u，端点v}
    for(int i = 0;i < m;i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        edges[i] = { w,u,v };
    }

    // 3. 按边权从小到大排序（tuple 默认比对第一个元素）
    sort(edges.begin(), edges.end());

    int res = 0; // 记录最小生成树总边权（原 ans）
    int cnt = 0; // 记录成功加入生成树的边数

    // 4. Kruskal 贪心加边
    for(int i = 0;i < m;i++)
    {
        auto [w, u, v] = edges[i]; // C++17 结构化绑定拆包
        u = find(u), v = find(v);
        if(u != v)
        {
            p[u] = v;
            res += w;
            cnt++; // 成功连边，计数器加 1
        }
    }

    // 5. 判断连通性
    // 原代码通过再跑一遍 O(N) 循环判所有点祖先是否相同。
    // 更优解法：一棵 n 个点的树必定刚好有 n-1 条边，直接判断 cnt 即可。
    if(cnt < n - 1)
    {
        cout << "orz" << endl; // 无法连通的情况
    }
    else
    {
        cout << res << endl;
    }
}
```

最小生成树 prim.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 5050, M = 2e5 + 10;
const int INF = 0x3f3f3f3f;

// 链式前向星存图
struct E {
    int v, w, x;
} e[M * 2]; // 无向图记得开两倍空间

int n, m, h[N], cnte;
int dis[N];     // 存储点到当前生成树集合的最小距离
bool vis[N];    // 标记点是否已加入生成树
int res = 0;    // 最小生成树权值和
int cnt = 0;    // 已加入生成树的点数量

void adde(int u, int v, int w) {
    e[++cnte] = E{ v, w, h[u] };
    h[u] = cnte;
}

struct S {
    int u, d;
```

```
    // 优先队列小根堆，距离小的优先
    bool operator<(const S& x) const {
        return d > x.d;
    }
};

priority_queue<S> q;

// 返回 true 表示成功生成 MST，false 表示图不连通
bool Prim() {
    // 初始化
    memset(dis, 0x3f, sizeof(dis));
    memset(vis, 0, sizeof(vis));
    res = 0;
    cnt = 0;

    // 从 1 号点开始
    dis[1] = 0;
    q.push({ 1, 0 });

    while(!q.empty()) {
        // 优化：如果已经选了 n 个点，说明生成树构建完成
        if(cnt >= n) break;

        int u = q.top().u;
        int d = q.top().d;
        q.pop();

        if(vis[u]) continue;
        vis[u] = 1;
        res += d;
        cnt++;

        for(int i = h[u]; i; i = e[i].x) {
            int v = e[i].v;
            int w = e[i].w;
            // 如果通过 u 到达 v 的距离比 v 当前记录的距离更短
            if(w < dis[v]) {
                dis[v] = w;
                q.push({ v, w });
            }
        }
    }

    // 判断是否所有点都在生成树内
    return cnt == n;
}

/* 使用示例：
 cin >> n >> m;
 for(int i=1; i<=m; i++){
     int u,v,w; cin>>u>>v>>w;
     adde(u,v,w); adde(v,u,w);
 }
 if(Prim()) cout << res << endl;
 else cout << "impossible" << endl;
*/
```

字符串

字符串-KMP

算法说明 * **用途**：在 $O(N+M)$ 的时间复杂度内，高效统计模式串 P 在主串 S 中出现的次数（或查找所有出现的位置）。

* **时间复杂度**：

* 预处理 ne 数组：$O(M)$，其中 M 为模式串长度。

* 匹配过程：$O(N)$，其中 N 为主串长度。

* **适用场景**：单模式串匹配问题、统计字符串出现频次、求字符串的最小循环节与公共前后缀。

kmp.cpp

```
#include<bits/stdc++.h>
using namespace std;

void solve()
{
    string s,p;
    cin>>s>>p;
    int n=s.size(),m=p.size();
    vector<int>ne(m+1);
    s=" "+s;
    p=" "+p;
    for(int i=2,j=0;i<=m;i++)
    {
        while(j&&p[i]!=p[j+1])j=ne[j];
        if(p[i]==p[j+1])j++;
        ne[i]=j;
    }
    int cn=0;
    for(int i=1,j=0;i<=n;i++)
    {
        while(j&&s[i]!=p[j+1])j=ne[j];
        if(s[i]==p[j+1])j++;
        if(j==m)
        {
            cn++;
            j=ne[j];
        }
    }
}
```

字符串 - Z 函数（扩展 KMP）

算法说明 * **用途**：在 $O(N)$ 时间内求出一个字符串所有后缀与前缀的最长公共前缀长度。

* **时间复杂度**：$O(N)$。

* **核心思想**：维护一个最靠右的匹配段 (Z-box) $[l,r]$，在该区间内的计算直接“白嫖”前缀 $z[i-l]$ 的值，并卡住边界，超出边界的部分再暴力扩展。

kmpz 函数.cpp

```
#include<bits/stdc++.h>
using namespace std;
void solve()
{
    string s;
    cin>>s;
    int n=s.size();
    vector<int>z(n);
```

```
    int l=0,r=0;
    for(int i=1;i<n;i++)
    {
        if(i<=r)z[i]=min(z[i-l],r-i+1);
        while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
        if(i+z[i]-1>r)
        {
            l=i;
            r=i+z[i]-1;
        }
    }

}
```

字符串 - Manacher 算法（马拉车）

算法说明 * **用途**：在 $O(N)$ 时间复杂度内，求出字符串中以每个位置为中心的最长回文子串的半径（从而找到全局最长回文子串）。

* **时间复杂度**：$O(N)$。

* **核心思想**：利用已经求出的大回文串的对称性，直接将左半边的答案镜像拷贝到右半边，避免重复扩展。

Manachar.cpp

```
#include<bits/stdc++.h>
using namespace std;
using PII = pair<int,int>;
#define int long long
#define endl '\n'
void solve()
{
    string s;
    cin>>s;
    string t="$#";
    for(auto c:s)
    {
        t+=c;
        t+="#";
    }
    t+='^';
    int n=t.size();
    vector<int>p(n+1);
    int C=0,R=0;//中心，最远R
    int mxle=0;
    for(int i=1;i<n-1;i++)
    {
        if(i<=R)p[i]=min(p[2*C-i],R-i+1);
        else p[i]=1;
        while(t[i-p[i]]==t[i+p[i]])p[i]++;
        if(i+p[i]-1>R)
        {
            C=i;
            R=i+p[i]-1;
        }
        if(p[i]-1>mxle)mxle=p[i]-1;
    }
    cout<<mxle<<endl;

}
```

数学

数论 - 整除分块（数论分块）

算法说明 * **用途**：快速计算形如 $\sum_{i=1}^{n}\lfloor\frac{k}{i}\rfloor$ 的区间取整求和问题。

* **时间复杂度**：$O(\sqrt{n})$。

* **适用场景**：适用于 n 极大（如 10^9 级别）导致 $O(n)$ 遍历超时的情况。利用 $\lfloor\frac{k}{i}\rfloor$ 取值成块连续相同的特性，每次直接计算并跳过一整段区间，极大提升效率。常作为莫比乌斯反演、杜教筛的底层组件。

整除分块.cpp

```
#include<bits/stdc++.h>
using namespace std;
void solve()
{
    int n;
    cin>>n;
    int ans=0;
    for(int l=0,r;l<=n;l=r+1)
    {
        r=n/(n/l);
        ans+=(r-l+1)*(n/l);
        l=r+1;
    }
}
```

矩阵与递推 - 矩阵快速幂

算法说明 * **用途**：快速求解线性递推数列的第 n 项（如斐波那契数列、方案数统计等），以及图论中求解定长路径的方案数。

* **时间复杂度**：$O(M^3\log k)$，其中 M 为矩阵的大小（状态维度），k 为递推的项数或幂次。

* **适用场景**：适用于递推项数极大（如 $k\geq 10^{18}$）且状态转移呈线性的场景。纯全局数组写法避开了 `struct` 和运算符重载的开销，结合了极其关键的 `if(!A[i][k]) continue;` 稀疏矩阵常数剪枝，运行极快。

矩阵快速幂.cpp

```
#include<bits/stdc++.h>
using namespace std;
const int N = 4;
const int MOD  = 1e9+10;

int a[N][N];
int res[N][N];

void mul(int c[][N],int A[][N],int B[][N])
{
    int tmp[N][N]={0};
    for(int i=0;i<N;i++)
    {
        for(int j=0;j<N;j++)
        {
            if(!A[i][j])continue;
            for(int k=0;k<N;k++)
            {
```

```
                tmp[i][k]=(tmp[i][k]+A[i][j]*B[j][k])%
                    MOD;
            }
        }

    }
    memcpy(c,tmp,sizeof tmp);
}

void qmi(int k)
{
    memset(res,0,sizeof res);
    for(int i=0;i<N;i++)res[i][i]=1;
    while(k)
    {
        if(k&1)mul(res,res,a);
        mul(a,a,a);
        k>>=1;
    }
}
//使用方法
int main()
{
    //1初始化底数，使得A*n=n+1
    a[0][0]=1;a[0][1]=1;
    a[1][0]=1;
    a[1][1]=0;
    qmi(10);
    cout<<res[0][0]<<endl;
    return 0;
}
```

组合数学 - 卡特兰数 (Catalan Number)

算法说明 * **用途**：用于解决“在特定规则下，两种操作数量始终满足偏序关系”的计数问题。

* **时间复杂度**：预处理阶乘与逆元 $O(N)$，单次查询 $O(1)$。

* **适用场景**：题目模型可转化为两种互相抵消的操作（如 $+1$ 和 -1），且限制任意时刻的前缀和必须 ≥ 0 时。核心是 **容斥原理 + 折线法（翻转法）**，推导出公式 $C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}$。

* **常见模型**：

1. **括号匹配**：n 对括号，合法的括号序列总数。
2. **出栈序列**：长度为 n 的序列依次进栈，不同的合法出栈序列总数。
3. **网格路径**：从 $(0,0)$ 到 (n,n)，只能向右或向上走，且不越过对角线 $y=x$ 的路径总数。
4. **二叉树形态**：n 个结点的二叉树，共有多少种不同的形态。
5. **多边形剖分**：凸 $n+2$ 边形通过对角线划分为三角形的方案数。
6. **找零问题**：n 个人拿 50 元，n 个人拿 100 元排队买票，售票员不会找不开钱的排队方案数。

组合数学-卡特兰数.cpp

```
#include <iostream>
using namespace std;

#define int long long
using PII = pair<int, int>;

const int MOD = 1e9 + 7;
const int MAXN = 2e5 + 5; // 计算 C_n 需要用到 2n 的阶乘，注意开两倍大小

int fact[MAXN], invFact[MAXN];

int qpow(int a, int b){
    int res = 1;
    while(b){
        if(b & 1)res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void init(){
    fact[0] = 1;invFact[0] = 1;
    for(int i = 1;i < MAXN;i++)fact[i] = fact[i - 1] *
        i % MOD;
    invFact[MAXN - 1] = qpow(fact[MAXN - 1], MOD - 2);
    for(int i = MAXN - 2;i >= 1;i--)invFact[i] =
        invFact[i + 1] * (i + 1) % MOD;
}

int C(int n, int m){
    if(m<0 || m>n)return 0;
    return fact[n] * invFact[m] % MOD * invFact[n - m]
        % MOD;
}

int get_catalan(int n){
    if(n < 0)return 0;
    return C(2 * n, n) * qpow(n + 1, MOD - 2) % MOD;
}
// O(1) 求广义卡特兰数：n 个操作A，m 个操作B，保证任意时刻 A的出现次数 >= B的出现次数
int get_generalized_catalan(int n, int m){
    if(n < m) return 0; // B操作比A操作多，绝对无法满足限制

    // Ans = C(n+m, m) - C(n+m, m-1)
    int total = C(n + m, m);
    int invalid = C(n + m, m - 1);

    // 注意取模时的减法防负数处理
    return (total - invalid + MOD) % MOD;
}
void solve(){
    int n;
    cin >> n;
    cout << get_catalan(n) << "\n";
}

signed main(){
    ios::sync_with_stdio(false);cin.tie(nullptr);
    init();
    int t = 1;
    cin >> t;
    while(t--)solve();
    return 0;
}
```

组合数学 - 多重集组合数

算法说明 * **用途**：有 k 种物品，第 i 种最多选 n_i 个，求一共选 r 个的方案数。
* **时间复杂度**：$O(2^k \cdot k)$。
* **适用场景**：种类 k 极小（$k \le 20$），总数 r 极大（无法用多重背包 DP）时。核心是 **容斥原理 + 隔板法**。

组合数学-多重组合数.cpp

```cpp
#include<bits/stdc++.h>>
using namespace std;
using namespace std;
#define int long long
const int MOD = 998244353;
const int N = 4e5+10;

int fa[N],infa[N];


int qmi(int a,int k)
{
    int res=1;
    while(k)
    {
        if(k&1)res=res*a%MOD;
        k>>=1;
        a=a*a%MOD;
    }
    return res;
}
void init()
{
    fa[0]=infa[0]=1;
    for(int i=1;i<N;i++)fa[i]=i*fa[i-1]%MOD;
    infa[N-1]=qmi(fa[N-1],MOD-2);
    for(int i=N-2;i>0;i--)infa[i]=infa[i+1]*(i+1)%MOD;
}

int C(int a,int b)
{
    if(a<0||b<0||b>a)return 0;
    return fa[a]*infa[b]%MOD*infa[a-b]%MOD;
}


int mulC(int k,int r,vector<int>&cn)
{
    int ans=0;
    for(int i=0;i<(1LL<<k);i++)
    {
        int lr=r;
        int pan=1;
        for(int j=0;j<k;j++)
        {
            if((i<<j)&1)
            {
                lr-=cn[j]+1;
                pan=-pan;
            }
        }
        if(lr<0)continue;
        ans=(ans+pan*C(lr+k-1,k-1)%MOD+MOD)%MOD;
    }
    return ans;
}
```

组合数学 - k 部分拆数 (k-Part Integer Partition)

算法说明 * **用途**：求将自然数 n 分拆为恰好 k 个正整数之和的方案数（不考虑顺序）。
* **时间复杂度**：全局预处理 $O(n \cdot k)$，单次查询 $O(1)$。
* **核心推导**：将分拆方案分为两类：
1. **至少包含一个 1**：将这个 1 拿走，等价于把 $n-1$ 拆分成 $k-1$ 个部分，方案数为 $p(n-1,k-1)$。
2. **所有部分都 ≥ 2**：将这 k 个部分每个都减去 1（总共减去 k），剩下的每个数依然 ≥ 1，等价于把 $n-k$ 拆分成 k 个部分，方案数为 $p(n-k,k)$。
* **状态转移方程**：$p(n,k) = p(n-1,k-1) + p(n-k,k)$

组合数学-拆分数.cpp

```cpp
#include <iostream>
#include <cstring>
using namespace std;


#define int long long
using PII = pair<int, int>;

const int MOD = 1000007;
const int MAXN = 10005;
const int MAXK = 1005;

int p[MAXN][MAXK]; // p[i][j] 表示将数字 i 拆成恰好 j 个部分的方案数

// 全局预处理，时间复杂度 O(N * K)
void init(){
    memset(p, 0, sizeof(p));
    p[0][0] = 1;
    for(int i = 1;i < MAXN;++i){
        for(int j = 1;j < MAXK;++j){
            if(i - j >= 0){
                // p[i-j][j]: 所有部分 >= 2
                // p[i-1][j-1]: 至少有一个部分是 1
                p[i][j] = (p[i - j][j] + p[i - 1][j - 1]) % MOD;
            }
        }
    }
}

void solve(){
    int n, k;
    // 多组输入处理
    if(cin >> n >> k){
        cout << p[n][k] << "\n";
    }
    else{
        exit(0);
    }
}
```

组合数学 - 互异分拆数 (Distinct Integer Partition)

算法说明 * **用途**：求将自然数 n 分拆为若干个**互不相同**的正整数之和的方案数。

* **时间复杂度**：利用分拆部分数的上限约为 $\sqrt{2n}$，时间复杂度降至 $O(n\sqrt{n})$。使用滚动数组优化，空间复杂度为 $O(n)$。

* **核心推导**：假设把 n 拆成 k 个互不相同的部分，将其从大到小排列 $r_1 > r_2 > \cdots > r_k \geq 1$。

给这 k 个数**同时减去 1**（总共减去 k），它们依然严格递减：

1. 如果原本最小的数 $r_k = 1$，减 1 后变成 0（消失了），剩下 $k-1$ 堆互不相同的数。方案数为 $pd(n-k, k-1)$。

2. 如果原本最小的数 $r_k \geq 2$，减 1 后 ≥ 1（没消失），依然是 k 堆互不相同的数。方案数为 $pd(n-k, k)$。

* **状态转移方程**：$pd(n,k) = pd(n-k, k-1) + pd(n-k, k)$

组合数学-拆分数互异.cpp

```cpp
#include <iostream>
#include <cstring>
using namespace std;

#define int long long
using PII = pair<int, int>;

const int MOD = 1000007;
const int MAXN = 50005;

// 使用滚动数组压缩 k 维度，pd[i][j&1] 表示将 i 拆为 j 个互异部分的方案数
int pd[MAXN][2];

void solve(){
    int n;
    if(!(cin >> n))exit(0);

    memset(pd, 0, sizeof(pd));
    pd[0][0] = 1;
    int ans = 0;

    // 互异分拆的极端情况是 1+2+...+k <= n。
    // 当 n=50000 时，k 的最大值约为 315，所以循环到 350 即可。
    for(int j = 1;j < 350;++j){
        // 每次进入新的一层 j，先清空当前层
        for(int i = 0;i <= n;++i)pd[i][j & 1] = 0;

        for(int i = 0;i <= n;++i){
            if(i - j >= 0){
                // pd[i-j][j&1]：抽底后原本最小的数依然 >=1（共 j 堆）
                // pd[i-j][(j-1)&1]：抽底后原本最小的数变成 0（剩 j-1 堆）
                pd[i][j & 1] = (pd[i - j][j & 1] + pd[i - j][(j - 1) & 1]) % MOD;
            }
        }
        // 将恰好拆成 j 个互异部分的方案数累加到最终答案
        ans = (ans + pd[n][j & 1]) % MOD;
    }
    cout << ans << "\n";
}
```

数学 - 斐波那契快速倍增法 (Fast Doubling)

算法说明 * **用途**：在 $O(\log N)$ 时间复杂度内，求出斐波那契数列的第 N 项对特定模数取模的结果，常数极小，全面碾压矩阵快速幂。

* **时间复杂度**：$O(\log N)$。

* **核心思想**：利用分治思想与恒等式 $F_{2k} = F_k(2F_{k+1} - F_k)$ 和 $F_{2k+1} = F_k^2 + F_{k+1}^2$，每次将所求项的下标减半，递归求解。

组合数学-斐波那.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
using PII = pair<int,int>;
#define int long long
const int MOD = 1e9+10;

PII fib(int n)
{
    if(n==0)return {0,1};
    auto p=fib(n>>1);
    int fk=p.first,fk1=p.second;
    int c=(2*fk1*fk%MOD-fk*fk%MOD+MOD)%MOD;
    int d=(fk*fk%MOD+fk1*fk1%MOD)%MOD;
    if(n&1)return {d,(c+d)%MOD};
    return {c,d};
}
//使用 fin(20)代表求第20项
```

数学 - 皮萨诺周期 (Pisano Period) 与超大数降维

算法说明 * **用途**：当 N 是高精度超大数（如长度达 10^5 的数字串）时，求 $F_N \pmod m$。通过计算模 m 下的周期 P，将原问题强行降维为 $F_{N \bmod P} \pmod m$。

* **时间复杂度**：求周期 $O(\sqrt{m})$，大数取模 $O(|N|)$，最后倍增求值 $O(\log m)$。

* **核心思想（周期分类查表法则）**：

斐波那契通项公式中包含 $\sqrt{5}$。在模质数 p 的世界里，$\sqrt{5}$ 是否存在（二次剩余）决定了周期的长度。利用二次互反律，我们只需看 p 除以 5 的余数（即个位特征）来查表：

1. **特例**：$p = 2$ 时基数为 3；$p = 5$ 时基数为 20。

2. **第一派** ($p \pmod 5 = 1$ **或** 4)：即个位为 1 或 9。此时模 p 意义下存在 $\sqrt{5}$，公式正常运转，周期基数取 $p-1$。

3. **第二派** ($p \pmod 5 = 2$ **或** 3)：即个位为 3 或 7。此时模 p 意义下不存在 $\sqrt{5}$，进入二维扩域，周期基数取 $2(p+1)$。

将 m 分解质因数后，分别计算上述基数并求最小公倍数 (LCM)，即可得到总周期。

组合数学-费波纳周期.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
using PII = pair<int,int>;
int MOD;

PII fib(int n)
{
```

```
    if(n==0)return {0,1};
    auto p=fib(n>>1);
    int fk=p.first,fk1=p.second;
    int c=fk*((2*fk1-fk+MOD)%MOD)%MOD;
    int d=(fk*fk%MOD+fk1*fk1%MOD)%MOD;
    if(n&1)return {d,(c+d)%MOD};
    return {c,d};
}

int qpow(int a,int k)
{
    int re=1;
    while(k)
    {
        if(k&1)re=a*re;
        a=a*a;
        k>>=1;
    }
    return re;
}

int lcm(int a,int b)
{
    return a/__gcd(a,b)*b;
}

int peropd(int m)
{
    int re=1,tem=m;
    for(int i=2;i*i<=tem;i++)
    {
        if(tem%i==0)
        {
            int p=i,e=0;
            while(tem%i==0)
            {
                tem/=i;
                e++;
            }
            int cur=qpow(p,e-1);
            if(p==2)cur*=3;
            else if(p==5)cur*=20;
            else if(p%5==1||p%5==4)cur*=(p-1);
            else cur*=2*(p+1);
            re=lcm(re,cur);
        }
    }
    if(tem>1)
    {
        int p=tem;
        int cur=1;
        if(p==2)cur*=3;
        else if(p==5)cur*=20;
        else if(p%5==1||p%5==4)cur*=(p-1);
        else cur*=2*(p+1);
        re=lcm(re,cur);
    }
    return re;
}
/*使用 求周期P=peropd( MOD)
知道周期后
int n_mod = 0;
    for(char c : n_str) {
        n_mod = (n_mod * 10 + (c - '0')) % P;
    }
    cout<<fib(n_mod).first
*/
```

组合数学 - 错位排列 (Derangement)

算法说明 * **用途**：计算 n 个元素的全错位排列方案数（没有任何一个元素在自己原来的位置上）。
* **时间复杂度**：预处理 $O(N)$，单次查询 $O(1)$。
* **核心思想**：动态规划/递推。考虑第 n 个元素放在位置 k，而第 k 个元素是否放在位置 n，可划分为两个子问题。
* **核心公式**：

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

组合数学-错排.cpp

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N = 1e6+10;
const int MOD = 1e9+7;
int d[N];
void pre()
{
    d[1]=0,d[2]=1;
    for(int i=3;i<N;i++)d[i]=(i-1)*(d[i-1]+d[i-2])%MOD
        ;
}
```

数论 - 线性筛质数 (欧拉筛)

算法说明 * **用途**：快速筛选出 $1 \sim n$ 范围内的所有质数。
* **时间复杂度**：$O(n)$。
* **适用场景**：适用于 n 较大（如 10^7 级别）的情况。相比埃氏筛，它保证每个合数只被其**最小质因子**筛去一次，因此是线性的，效率极高。

质数筛.cpp

```
const int N = 1e7 + 10;
int pri[N], cnt;
bool st[N];
void getpri(int n)
{
    for(int i = 2;i <= n;i++)
    {
        if(!st[i])pri[cnt++] = i;
        for(int j = 0;pri[j] <= n / i;j++)
        {
            st[pri[j] * i] = 1;
            if(i % pri[j] == 0)break;
        }
    }
}
```

数据结构

数据结构 - ST 表 (RMQ)

算法说明 * **用途**：解决静态数组的“可重复贡献问题”区间查询（如最值、GCD 等），不支持修改。

* **时间复杂度**：

* 预处理：$O(N \log N)$。

* 单次查询：$O(1)$。

* **适用场景**：数组无动态修改且查询极频繁（$M \geq 10^6$）时，$O(1)$ 速度平替线段树。

* **注意事项**：

1. `pre()` 预处理 `logn` 必须在 `main` 最开头仅调用一次，严防多测 TLE。

2. 建表外层倍增循环界限写为 `(1<<j) <= n` 进行常数级剪枝。

3. 查询区间 $[L, R]$ 时，步长 $s = \log n[R - L + 1]$，结果由两端覆盖：$\max(f[L][s], f[R - (1 \ll s) + 1][s])$。

st 表.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 2e5+10;
const int logN = 22;//LOG(n)
int f[N][logN],logn[N];

void pre()
{

    logn[1]=0;
    logn[2]=1;
    for(int i=3;i<N;i++)logn[i]=logn[i/2]+1;
}

void solve()
{
    int n;
    cin>>n;
    for(int i=1;i<=n;i++)cin>>f[i][0];
    pre();
    for(int j=1;(1<<j)<=n;j++)
    {
        for(int i=1;i+(1<<j)-1<=n;i++)
        {
            f[i][j]=max(f[i][j-1],f[i+(1<<(j-1))][j
                -1]);
        }
    }
    int l,r;
    cin>>l>>r;
    int s=logn[r-l+1];
    cout<<max(f[l][s],f[r-(1<<s)+1][s])<<endl;
}
```

分块 - 区间加 & 区间秩查询

算法说明 * **用途**：在 $O(\sqrt{N} \log \sqrt{N})$ 的时间内维护序列的区间增量修改，并支持查询区间内小于特定值 x 的元素个数。

* **时间复杂度**：

* 建块：$O(N \log \sqrt{N})$。

* 区间修改：$O(\sqrt{N} \log \sqrt{N})$。

* 区间查询：$O(\sqrt{N} \log \sqrt{N})$。

* **适用场景**：处理无法直接通过线段树或树状数组维护的非线性区间问题（如区间秩、区间众数、区间第 k 大），常用于处理带区间加法的统计问题。

分块.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define endl '\n'

const int N = 1e5 + 10, BSI = 400;

int a[N], sora[N], lazy[BSI];
int L[BSI], R[BSI], pos[N];
int n, q, blosize, blonum;
// 重构函数：散块修改后同步 sora 镜像并排序
void rebuild(int id)
{
    for (int i = L[id]; i <= R[id]; i++)sora[i] = a[i
        ];
    sort(sora + L[id], sora + R[id] + 1);
}

// 操作 1：区间增量
void update(int l, int r, int x)
{
    int bl = pos[l], br = pos[r];
    if (bl == br)
    {
        for (int i = l; i <= r; i++) a[i] += x;
        rebuild(bl);
    }
    else
    {
        // 左散块处理
        for (int i = l; i <= R[bl]; i++) a[i] += x;
        rebuild(bl);
        // 中间整块处理标记
        for (int i = bl + 1; i <= br - 1; i++) lazy[i]
            += x;
        // 右散块处理
        for (int i = L[br]; i <= r; i++) a[i] += x;
        rebuild(br);
    }
}
// 操作 2：区间小于 x 计数
int query(int l, int r, int x)
{
    int bl = pos[l], br = pos[r];
    int ans = 0;
    if (bl == br)
    {
        for (int i = l; i <= r; i++)
        {
            if (a[i] + lazy[bl] < x) ans++;
        }
    }
    else
    {
```

```
        // 左右散块暴力匹配
        for (int i = l; i <= R[bl]; i++) if (a[i] +
            lazy[bl] < x) ans++;
        for (int i = L[br]; i <= r; i++) if (a[i] +
            lazy[br] < x) ans++;
        // 中间整块二分查找
        for (int i = bl + 1; i <= br - 1; i++)
        {
            int target = x - lazy[i];
            ans += lower_bound(sora + L[i], sora + R[i
                ] + 1, target) - (sora + L[i]);
        }
    }
    return ans;
}

void solve()
{
    if (!(cin >> n >> q)) return;
    blosize = sqrt(n);
    blonum = (n - 1) / blosize + 1;
    for (int i = 1; i <= n; i++)
    {
        cin >> a[i];
        pos[i] = (i - 1) / blosize + 1;
        sora[i] = a[i];
    }
    // 初始化边界并对块内进行首次预排序
    for (int i = 1; i <= blonum; i++)
    {
        L[i] = (i - 1) * blosize + 1;
        R[i] = min(i * blosize, n);
        sort(sora + L[i], sora + R[i] + 1);
    }

    while (q--)
    {
        int op, l, r, x;
        cin >> op >> l >> r >> x;
        if (op == 1) update(l, r, x);
        else cout << query(l, r, x) << endl;
    }
}
```

树 - 最近公共祖先 (LCA) - 倍增法

算法说明 * **用途**：在有根树中，快速查询两个节点的最近公共祖先。

* **时间复杂度**：

* 预处理：$O(N \log N)$。

* 单次查询：$O(\log N)$。

* **适用场景**：适用于 N, M 较大（如 $10^5 \sim 10^6$）且需要在线查询的情况。

* **注意事项**：

1. f 数组第二维的大小由 N 决定，通常 $2^{19} > 5 \times 10^5$，所以开 20 足够。

2. 利用 de[0] = 0 (全局变量默认值) 作为哨兵，防止倍增跳越界。

树-LCA 倍增算法.cpp

```
#include<bits/stdc++.h>
using namespace std;

const int N = 500010;

vector<int> g[N];

int f[N][20];  // f[u][i] 表示 u 的第 2^i 个祖先
int de[N]; //深度数组，0号点深度为0作为哨兵
/**
 * DFS 预处理倍增数组和深度
 * @param u 当前节点
 * @param fa 父节点 (根节点调用时 fa 传 0)
 */
void dfs(int u, int fa) {
    de[u] = de[fa] + 1;
    f[u][0] = fa;

    // 倍增核心逻辑
    for(int i = 1; i < 20; i++) {
        f[u][i] = f[f[u][i - 1]][i - 1];
    }
    for(int v : g[u])
    {
        if(v == fa) continue;
        dfs(v, u);
    }
}

/**
 * 查询 LCA
 */
int lca(int a, int b) {
    if(de[a] < de[b]) swap(a, b);
    // 1. 将 a 跳到和 b 同一深度
    for(int i = 19; i >= 0; i--) {
        if(de[f[a][i]] >= de[b]) a = f[a][i];
    }
    if(a == b) return a;
    // 2. a 和 b 一起向上跳，直到父亲相同
    for(int i = 19; i >= 0; i--) {
        if(f[a][i] != f[b][i]) {
            a = f[a][i];
            b = f[b][i];
        }
    }
    return f[a][0]; // 返回父节点
}
int main()
{
    return 0;
}
```

树 - 笛卡尔树

算法说明 * **用途**：将序列转化为同时满足“二叉搜索树（按下标）”和“堆（按权值）”性质的二叉树。

* **时间复杂度**：

* 建树：$O(N)$。

* **适用场景**：将序列区间问题转化为树上问题（如 RMQ 转 LCA），或求解直方图最大矩形/子段问题。本质是单调栈的树形化。

* **注意事项**：

1. 多组数据时**务必清空单调栈指针** `top = 0` 及左右儿子数组 `ls, rs`。
2. 单调栈 `st` 中存储的是数组的**下标**，比较大小时需代入原数组判定（如 `ar[st[top]] > ar[i]`）。
3. 线性建树完毕后，整棵树的根节点固定为栈底元素 `st[1]`。

树-笛卡尔树.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 10;
vector<int>ar(N), ls(N), rs(N);
vector<int>st(N);
int top;
int build(int n)
{
    top = 0;
    for(int i = 1;i <= n;i++)ls[i] = rs[i] = 0;
    for(int i = 1;i <= n;i++)
    {
        int la = 0;
        while(top && ar[st[top]] > ar[i])
        {
            la = st[top];
            top--;
        }
        if(top)rs[st[top]] = i;
        ls[i] = la;
        st[++top] = i;
    }
    return st[1];
}
```

数据结构 - 懒标记线段树

算法说明 * **用途**：维护一个长度为 N 的数组，支持单次将区间 $[L, R]$ 内的所有元素加上 v，以及查询区间 $[L, R]$ 内的最大值。

* **时间复杂度**：建树 $O(N)$，单次区间修改 $O(\log N)$，单次区间查询 $O(\log N)$。

* **适用场景**：需要频繁交替进行“区间修改”和“区间查询”时（普通差分数组无法处理边修改边查询的情况）。核心是 **分治思想 + 延迟更新（Lazy Tag）**。s

线段树-懒标记线段树.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
const int N = 1e5+10;

int w[N];
int tr[N<<2];
int add[N<<2];
void pushup(int u)
{
    tr[u]=max(tr[u<<1],tr[u<<1|1]);
}
void pushdown(int u,int l,int r)
{
    if(add[u])
    {
        tr[u<<1]+=add[u];
        tr[u<<1|1]+=add[u];
        add[u<<1]+=add[u];
        add[u<<1|1]+=add[u];
        add[u]=0;
    }
}

void build(int u,int l,int r)
{
    if(l==r)
    {
        tr[u]=w[l];
        return ;
    }
    int mid=l+r>>1;
    build(u<<1,l,mid);
    build(u<<1|1,mid+1,r);
    pushup(u);
}

void modify(int u,int l,int r,int L,int R,int v)
{
    if(L<=l&&r<=R)
    {
        tr[u]+=v;
        add[u]+=v;
        return ;
    }
    pushdown(u,l,r);
    int mid=l+r>>1;
    if(L<=mid)modify(u<<1,l,mid,L,R,v);
    if(R>mid)modify(u<<1|1,mid+1,r,L,R,v);
    pushup(u);
}

int query(int u,int l,int r,int L,int R)
{
    if(l>=L&&r<=R)return tr[u];
    pushdown(u,l,r);
    int mid=l+r>>1;
    int res=-4e18;
    if(L<=mid)res=max(res,query(u<<1,l,mid,L,R));
    if(R>mid)res=max(res,query(u<<1|1,mid+1,r,L,R));
    return res;
}

// //1初始化原数组 1 2 3 4 5
// for(int i=1;i<=n;i++)w[i]=i;
// //2建树
// build(1,1,n);
// //3区间修改：给区间 [2,4] 加上 10
// modify(1,1,n,2,4,10);
// //4区间查询：输出区间 [1,5] 的max
// cout<<query(1,1,n,1,5)<<endl;
```

杂项

数据结构 - 珂朵莉树 (ODT / Old Driver Tree)

算法说明 * **用途**：维护序列的区间操作。擅长处理具有 **“区间推平（区间赋值）”** 操作且数据有一定随机性的题目。

* **时间复杂度**：在有大量随机区间赋值操作的前提下，set 中的节点数会被极大地压缩，所有区间操作的均摊时间复杂度趋近于 $O(m \log n)$。若缺乏推平操作，最坏会退化到 $O(mn)$。
* **核心思想**：利用 std::set 维护一段段值相同的连续区间。核心操作只有两个：split（将一个大区间按指定位置劈开）和 assign（将一段区间内的所有碎块删掉，替换成一整块）。
* **避坑铁律**：在执行区间操作时，**必须先 split(r + 1)，然后再 split(l)**，否则可能会导致右侧的迭代器失效，引发 RE（段错误）。

珂朵莉树.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

struct ODT{
    int l,r;
    mutable int v;//允许我们在不删节点的情况下直接修改 set 内部元素的值
    bool operator<(const ODT& x)const{
        return l<x.l;
    }
};
set<ODT>odt;
auto split(int pos)
{
    auto it=odt.lower_bound({pos,0,0});
    if(it!=odt.end()&&it->l==pos)return it;
    --it;
    int l=it->l,r=it->r;
    int v=it->v;
    odt.erase(it);
    odt.insert({l,pos-1,v});
    return odt.insert({pos,r,v}).first;
}
void assign(int l,int r,int v)
{
    auto itr=split(r+1);
    auto itl=split(l);
    odt.erase(itl,itr);
    odt.insert({l,r,v});
}

void init(int n)
{
    odt.clear();
    odt.insert({1,n,0});
}

// 扩展：区间加法
void add(int l, int r, long long val) {
    auto itr = split(r + 1);
    auto itl = split(l);
    for (auto it = itl; it != itr; ++it) {
        it->v += val; // mutable 发挥作用，直接修改值
    }
}

// 扩展：区间统计 / 暴力遍历
// 统计区间 [l, r] 内有多少个满足某种条件的值
long long query(int l, int r) {
    auto itr = split(r + 1);
    auto itl = split(l);
    long long res = 0;
    for (auto it = itl; it != itr; ++it) {
        // it->r - it->l + 1 就是该碎块的长度
        // 可以在这里做任何暴力统计，比如求和：res += (it->r - it->l + 1) * it->v;
    }
    return res;
}
```